

# Apache Spark

Distributed processing, SQL, streaming, performance, and production fundamentals

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

## 1. Apache Spark: What It Is and Why It Exists

Apache Spark is a distributed data-processing engine designed to process large datasets across clusters. It provides APIs for Python, Scala, Java, and R and includes libraries for SQL, streaming, machine learning, and graph processing.

**Key idea:** Spark is an execution engine, not a database. Data may live in object storage, HDFS, files, or databases while Spark supplies distributed computation.

### Core problem

A single machine can become constrained by CPU, memory, storage bandwidth, or the amount of data that must be processed. Distributed processing divides work across multiple machines and executes pieces in parallel.

Spark tries to make distributed computation accessible through higher-level APIs while still exposing controls for partitioning, caching, joins, and execution.

- Parallelism comes from partitions: a dataset is divided into logical pieces that can be processed concurrently.
- A Spark application normally has a driver plus executors that perform tasks.
- The same logical operation can be expressed with DataFrames, SQL, RDDs, or higher-level libraries.

### Major components

Spark Core provides the execution foundation. Spark SQL provides structured processing. Structured Streaming extends the structured APIs to streaming data. MLlib provides scalable machine-learning algorithms. GraphX provides graph-processing capabilities.

For data engineering, Spark SQL/DataFrames and Structured Streaming are usually the most important interfaces.

- Spark Core: scheduling, task execution, memory management, and fault tolerance.
- Spark SQL: SQL queries, DataFrames, schemas, Catalyst optimization, and Tungsten execution improvements.
- Structured Streaming: incremental processing over streaming sources.
- MLlib: distributed machine-learning algorithms and utilities.

### Mental model

Think of a Spark job as a logical computation that Spark turns into stages and tasks. The driver coordinates the application; executors run tasks and hold cached data.

The important distinction is between what the program asks Spark to do and how Spark physically executes it.

- Learn the logical plan first, then inspect the physical plan when performance matters.
- A transformation usually builds a plan; an action causes Spark to execute the plan.
- Network data movement, called a shuffle, is often much more expensive than local computation.

## 2. Spark Architecture and Execution

Spark applications use a driver process and executor processes. A cluster manager allocates resources, while Spark schedules work as tasks over partitions.

**Key idea:** The driver plans and coordinates; executors execute tasks. Understanding this separation is essential for diagnosing failures and performance problems.

### Driver

The driver runs the application's main program and creates the `SparkSession/SparkContext`. It builds execution plans, requests resources, schedules tasks, and tracks progress.

- Do not collect huge datasets to the driver; driver memory is not cluster memory.
- Driver-side loops over large distributed data often defeat Spark's distributed design.
- The driver should mainly coordinate rather than perform bulk data processing.

### Executors

Executors are long-lived processes for a Spark application. They run tasks and can cache data in memory or local storage.

- Executor CPU and memory determine how much work can run concurrently.
- Executor loss can cause cached data to be recomputed when lineage permits.
- Executor logs are valuable when diagnosing task failures, memory pressure, or serialization problems.

### Tasks, stages, jobs

An action creates a job. Spark divides the job into stages based largely on shuffle boundaries, and stages contain tasks that operate on partitions.

- One task generally processes one partition for a stage.
- A stage with 500 partitions can expose up to 500 tasks for that stage.
- Wide dependencies commonly introduce stage boundaries because data must move between executors.

### Cluster managers

Spark can run with different resource managers, including standalone mode, Kubernetes, and YARN. Managed cloud services add their own control-plane abstractions.

- The Spark programming model remains similar even when the infrastructure changes.
- Production choices depend on security, operations, ecosystem integration, and workload requirements.

### 3. RDDs: The Original Distributed Abstraction

An RDD (Resilient Distributed Dataset) is an immutable, partitioned collection of records that can be processed in parallel. RDDs remain important for understanding Spark's foundations even though DataFrames are usually preferred for structured workloads.

**Key idea:** RDDs expose distributed collections and lineage directly; DataFrames expose structured data and allow Spark to optimize more aggressively.

#### Properties

RDDs are immutable and partitioned. Transformations create new RDDs rather than changing existing ones. Spark can reconstruct lost partitions using lineage in many cases.

- Resilient: fault recovery can use lineage.
- Distributed: partitions are spread across executors.
- Dataset: the abstraction represents a collection of records.

#### Transformations

Examples include map, filter, flatMap, mapPartitions, union, distinct, and reduceByKey. Transformations are generally lazy.

- map changes each element independently.
- filter removes elements that do not satisfy a condition.
- reduceByKey performs aggregation while allowing map-side combining.
- mapPartitions works at partition granularity and can reduce per-record setup overhead.

#### Actions

Actions request a result or external effect, such as count, collect, take, reduce, saveAsTextFile, or writing a table.

- collect should only be used when the result is known to fit safely in driver memory.
- take and limit are useful for inspection.
- Writing output is an action because Spark must execute the upstream plan.

#### Lineage and fault tolerance

Spark records how an RDD was derived. If a partition is lost, Spark can often recompute it from the lineage rather than replicating every intermediate result.

- Lineage is one reason Spark can recover from executor failures.
- Checkpointing can truncate long lineage chains in workloads such as streaming or iterative algorithms.
- Persisting can avoid repeating expensive upstream computation when the same data is reused.

## 4. DataFrames, Spark SQL, and Schemas

A DataFrame is a distributed table-like structure with named columns and a schema. Spark SQL lets the optimizer reason about structured operations and often produce better execution plans than low-level record processing.

**Key idea:** For most modern data-engineering workloads, start with DataFrames/Spark SQL unless a specific requirement makes RDD-level control necessary.

### Schema

A schema describes column names and data types. Explicit schemas improve correctness, documentation, and predictable ingestion compared with always relying on inference.

- Common types include string, integer, long, double, decimal, boolean, date, timestamp, arrays, maps, and structs.
- Nullability is part of schema information, but actual source data can still violate expectations.
- Schema evolution should be treated as a data-contract concern.

### DataFrame operations

Common operations include select, filter/where, withColumn, groupBy, agg, join, orderBy, union, drop, and window functions.

- Prefer expressions over Python loops for distributed transformations.
- Use column expressions so Spark can optimize and push work toward the data source where possible.
- Avoid repeatedly adding columns with complex chains when a clearer select expression can express the transformation.

### Spark SQL

SQL queries can operate over registered tables and views. SQL and DataFrame APIs generally compile to a shared logical representation.

- SQL is useful for analysts and data engineers who think naturally in relational operations.
- DataFrame APIs are useful when transformations are constructed programmatically.
- The choice is often style and team preference rather than a fundamental engine difference.

### Catalyst and execution

Spark SQL analyzes logical plans, applies optimization rules, and generates physical plans. This lets Spark optimize operations such as predicate pushdown, column pruning, join selection, and expression simplification.

- Use `explain()` to inspect plans.
- Optimization is constrained by what Spark can understand; opaque Python logic can reduce optimization opportunities.
- Data source capabilities determine how much filtering and projection can be pushed down.

## 5. Lazy Evaluation, Transformations, Actions, and Shuffles

Spark transformations are generally lazy: they describe work without immediately executing it. An action triggers execution. This lets Spark build a plan before deciding how to run it.

**Key idea:** The cost of a Spark statement is often not visible at the line where it is written. The expensive work may occur when a later action triggers the complete lineage.

### Narrow vs wide transformations

A narrow transformation can compute an output partition from a small number of input partitions without moving records across the cluster. A wide transformation requires data redistribution.

- filter and many map operations are narrow.
- groupByKey, many joins, distinct, and orderBy commonly require shuffles.
- Shuffle boundaries often create new stages.

### Shuffle

A shuffle redistributes records across executors so that related records can be processed together. It uses network, disk, serialization, and CPU resources.

- Shuffles are a major source of latency.
- Large shuffles can spill to disk.
- Skew can cause one or a few tasks to process far more data than others.

### Why laziness helps

Spark can combine compatible operations, eliminate unnecessary work, and choose physical strategies after seeing the complete logical plan.

- Filter early when it reduces data substantially.
- Select only required columns.
- Avoid triggering multiple actions that recompute the same expensive lineage when caching is appropriate.

### Practical debugging

Use Spark UI and explain plans to identify stages, task durations, shuffle read/write, input/output sizes, and skew.

- A stage with one very slow task often suggests skew or a problematic partition.
- Repeated jobs over the same source may indicate missing persistence.
- A huge number of tiny tasks can indicate over-partitioning or small input files.

## 6. Partitioning, Joins, Aggregations, and Skew

Partitioning determines how data is distributed. Joins and aggregations often require data to be colocated by key, making partition strategy central to Spark performance.

**Key idea:** A good partitioning strategy reduces unnecessary data movement while keeping work balanced.

### Partition count

Too few partitions can underuse the cluster; too many can create scheduling overhead and tiny tasks. The appropriate number depends on data size, cluster resources, file layout, and workload.

- Inspect actual partition sizes rather than relying on a universal number.
- Repartition usually causes a shuffle.
- Coalesce can reduce partitions with less movement in suitable cases.

### Join strategies

Spark may use broadcast hash joins, sort-merge joins, shuffle hash joins, or other strategies depending on statistics, configuration, and data shape.

- Broadcast a genuinely small dimension table to avoid shuffling the large side.
- Sort-merge joins are common for large equi-joins.
- Join performance depends on key distribution, statistics, and partitioning.

### Data skew

Skew occurs when a small number of keys account for a disproportionate amount of data. One task may become much slower than the others.

- Find top key frequencies before large joins when skew is suspected.
- Salting can split hot keys into multiple logical keys when appropriate.
- Adaptive Query Execution can mitigate some skewed-join situations.

### Aggregations

Prefer aggregations that can combine data locally before shuffle, such as `reduceByKey`-style patterns or optimized SQL aggregations.

- Avoid `groupByKey` when an aggregation can be expressed as `reduceByKey` or an equivalent `DataFrame` aggregation.
- Use approximate algorithms where exactness is unnecessary and supported.

## 7. Spark Performance and Production Tuning

Spark performance is primarily about minimizing unnecessary data, movement, serialization, and repeated computation while using cluster resources effectively.

**Key idea:** Measure first. Spark tuning without looking at the execution plan and Spark UI often turns into guesswork.

### Reduce data early

Push filters and projections toward the source. Read only the columns and rows required by the job. Avoid materializing huge intermediate datasets.

- Partition pruning can reduce files read.
- Column pruning reduces I/O and memory.
- Predicate pushdown can let a data source perform filtering.

### Caching and persistence

Caching is useful when the same computed dataset is reused across multiple actions or branches. It is not a default requirement for every DataFrame.

- Cache only when reuse justifies memory/storage cost.
- Unpersist data that is no longer needed.
- Choose storage levels based on memory and recomputation trade-offs.

### File layout

Large numbers of tiny files can cause overhead during planning and execution. Extremely large files can also reduce parallelism or complicate recovery.

- Aim for reasonably sized files appropriate to the storage system and workload.
- Compaction can improve downstream read performance.
- Partition directory design should reflect common filter patterns, not every possible column.

### Adaptive Query Execution

AQE can use runtime statistics to improve physical execution, including changing join strategies, coalescing shuffle partitions, and handling some skewed joins.

- Keep AQE enabled where supported unless a measured reason exists to change it.
- Inspect the final plan rather than assuming the initial plan is the executed plan.

### Operational checklist

Monitor failed tasks, executor memory, GC, shuffle, input/output, and stage duration. Treat data quality and schema drift as production concerns alongside performance.

- Set realistic resource limits.
- Make jobs idempotent when possible.
- Log input versions, row counts, and output locations for reproducibility.

## 8. Structured Streaming

Structured Streaming provides a high-level streaming model built on Spark SQL/DataFrames. A streaming query continuously processes data from a source and writes results to a sink.

**Key idea:** Streaming is not simply 'Spark running forever'. Correctness depends on checkpointing, output mode, watermarks, event-time semantics, and the behavior of the source and sink.

### Core concepts

A streaming DataFrame represents an unbounded table. The query describes how each arriving batch of data contributes to the result.

- Sources can include files, Kafka, and other supported connectors.
- Sinks can include files, tables, Kafka, and other supported destinations.
- A checkpoint stores progress and state needed for recovery.

### Event time and processing time

Event time is when an event actually happened; processing time is when Spark handles it. Event-time processing is important when events arrive late or out of order.

- Watermarks tell Spark how long to wait for late data in supported stateful operations.
- Watermarks also help bound state retention.
- Choosing a watermark is a business/data-latency trade-off.

### Output modes

Append emits new rows that are finalized for the query; update emits changed rows; complete can output the full result for supported aggregations.

- The compatible output mode depends on the query.
- Sink semantics matter: at-least-once behavior and idempotency should be considered.

### State and reliability

Stateful operations such as aggregations and stream-stream joins require state management. Checkpointing is essential for fault recovery and progress tracking.

- Design for restartability.
- Monitor state size and processing latency.
- Avoid unbounded state when a bounded business rule can be used.



## 9. Spark MLlib and Data Engineering Patterns

MLlib provides distributed machine-learning algorithms and utilities. Even when your main goal is data engineering, understanding the interface helps you recognize where feature engineering and model pipelines fit.

**Key idea:** A data engineer often prepares reliable, well-partitioned, quality-controlled features before a model is trained.

### MLlib concepts

MLlib includes classification, regression, clustering, recommendation, feature engineering, evaluation, and pipeline APIs. The DataFrame-based API is the main modern interface.

- Transformers convert DataFrames to new DataFrames.
- Estimators learn parameters from data and produce transformers.
- Pipelines chain stages into repeatable workflows.

### Feature engineering

Feature engineering converts raw columns into model-ready representations. Typical operations include encoding categorical variables, scaling numerical features, tokenization, and vector assembly.

- Keep feature definitions versioned and reproducible.
- Prevent training/serving leakage by separating information available at training time from future information.
- Track the source and transformation logic of important features.

### Evaluation

Different tasks require different metrics. Classification can use precision, recall, F1, ROC-AUC, or PR-AUC; regression can use MAE, RMSE, and R-squared depending on the objective.

- Metric selection should reflect business cost, not only mathematical convenience.
- Use separate validation/test data appropriately.
- Distributed evaluation still needs careful sampling and statistical reasoning.

### Data-engineering connection

Reliable ingestion, schema management, deduplication, temporal correctness, and data quality are prerequisites for trustworthy ML.

- Treat feature pipelines as production data pipelines.
- Monitor feature freshness and distribution changes.
- Keep raw and curated layers separate when auditability matters.

## 10. Spark Production Checklist and Interview Map

This page consolidates the Spark concepts most useful for building pipelines and explaining design decisions in interviews.

**Key idea:** If you can explain partitions, stages, tasks, shuffles, joins, caching, DataFrames, and Structured Streaming, you understand the core of Spark data engineering.

### Architecture questions

Be able to explain the driver, executors, cluster manager, tasks, stages, jobs, partitions, and shuffle boundaries.

- Why is collect dangerous?
- What causes a shuffle?
- Why can one task be much slower than the others?
- When would you broadcast a table?

### DataFrame/SQL questions

Explain schema, select/filter, groupBy/agg, joins, windows, views, and explain plans.

- Why prefer DataFrames over RDDs for structured data?
- What is predicate pushdown?
- What is column pruning?
- How does Spark choose a physical join strategy?

### Production questions

Discuss idempotency, checkpointing, monitoring, partition sizing, small files, data quality, retries, and cost.

- Avoid magic tuning numbers.
- Measure with Spark UI and execution plans.
- Design outputs so reruns do not silently duplicate data.
- Keep credentials and secrets outside source code.

### Learning sequence

A practical sequence is: DataFrames → SQL → execution plans → partitioning → joins/shuffles → performance → Structured Streaming → production orchestration.

- Use small datasets to understand behavior before using large clusters.
- Build the RAG system in layers so each component can be tested independently.

### Selected official sources

- <https://spark.apache.org/docs/latest/>
- <https://spark.apache.org/docs/latest/rdd-programming-guide.html>
- <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- <https://spark.apache.org/docs/latest/ml-guide.html>